Artificial Intelligence
UNIT IV
Knowledge

Prepared By

Mrs. Priyadarshani Doke

First-Order Logic

First-order logic

- First-order logic (FOL) models the world in terms of
  - **Objects,** which are things with individual identities
  - **Properties** of objects that distinguish them from other objects
  - **Relations** that hold among sets of objects
  - **Functions,** which are a subset of relations where there is only one "value" for any given "input"
- Examples:
  - **Objects:** Students, lectures, companies, cars ...
  - Relations: Brother-of, bigger-than, outside, part-of, has-color, occurs-after, owns, visits, precedes, ...
  - Properties: blue, oval, even, large, ...
  - Functions: father-of(), best-friend(), third inning of ()…

User provides

- **Constant symbols,** which represent individuals in the world
  - Mary
  - 3
  - Green
- **Function symbols,** which map individuals to individuals
  - father-of(Mary) = John
  - color-of(Sky) = Blue
- **Predicate symbols,** which map individuals to truth values
  - greater(5,3)
  - green(Grass)
  - color(Grass, Green)

FOL Provides

- **Variable symbols**
  - E.g., x, y
- **Connectives**
  - Same as in PL: not ($\neg$), and ($\wedge$), or ($\vee$), implies ($\rightarrow$), if and only if (biconditional $\leftrightarrow$)
- **Quantifiers**
  - Universal $\forall$ **x** or **(Ax)**
  - Existential $\exists$ **x** or **(Ex)**

Sentences are built from terms and atoms

- A **term** (denoting a real-world individual) is a constant symbol, a variable symbol, or an n-place function of n terms.

  x and $f(x_1, ..., x_n)$ are terms, where each x_i is a term.

  A term with no variables is a **ground term**

- An **atomic sentence** (which has value true or false) is an n-place predicate of n terms. An atomic sentence is **an atomic formula containing no variables**. It follows that an atomic sentence contains no logical connectives, variables, or quantifiers.

- let F, G, H be predicate letters;
- let a, b, c be individual constants;
- let x, y, z be variables.

Atomic sentences

An atomic sentence is a simple **declarative sentence** which can either be true or false.

We write atomic sentences in the blocks language by combining a predicate (which always begins with a capital letter), followed by (in parentheses) one or more individual constants (which always begin with a lower case letter).

Ex:- Cube(a), Larger(b,c)

These wffs are atomic sentences; they contain no free variables or conjunctions:

- $F(a)$
- $G(a, b)$
- $H(a, b, c)$

Ex:- Claire gave Scruffy to Max. Can be denote:- GaveScruffy(claire, max).

Atomic formula

An **atomic formula** or **atom** is simply a predicate applied to a tuple of terms; that is, an atomic formula is a formula of the form $P(t_1, \ldots, t_n)$ for P a predicate, and the t_n terms.

These wffs are atomic formulae, but are not sentences (atomic or otherwise) because they include free variables:

- $F(x)$
- $G(a, z)$
- $H(x, y, z)$

Formulas are built from atomic formulas using logical connectives and quantifiers. Every atomic formula is a formula. If A and B are formulas and x is a variable then (A), $\neg$A, A $\wedge$ B, A $\vee$ B, A$\Rightarrow$ B, ($\forall$ x)A, and ($\exists$ x)A are formulas.

Sentences are built from terms and atoms (Cont…)

- A **complex sentence** is formed from atomic sentences connected by the logical connectives:

  $\neg P$, $P \vee Q$, $P \wedge Q$, $P \rightarrow Q$, $P \leftrightarrow Q$ where P and Q are sentences

- A **quantified sentence** adds quantifiers $\forall$ and $\exists$

- A **well-formed formula (wff)** is a sentence containing no "free" variables. That is, all variables are "bound" by universal or existential quantifiers.

  $(\forall x)P(x)$ has x bound as a universally quantified variable

  $(\forall x)P(x,y)$ has x bound as a universally quantified variable, but y is free.

Quantifiers

- **Universal quantification**
  - $(\forall x)P(x)$ means that P holds for **all** values of x in the domain associated with that variable
  - E.g., $(\forall x)$ dolphin(x) $\rightarrow$ mammal(x)
- **Existential quantification**
  - $(\exists x)P(x)$ means that P holds for **some** value of x in the domain associated with that variable
  - E.g., $(\exists x)$ mammal(x) $\wedge$ lays-eggs(x)
  - Permits one to make a statement about some object without naming it

Quantifiers

- Universal quantifiers are often used with "implies" to form "rules":
  $(\forall x)$ student(x) $\rightarrow$ smart(x) means "All students are smart"

- Universal quantification is *rarely* used to make blanket statements about every individual in the world:
  $(\forall x)$student(x) $\wedge$ smart(x) means "Everyone in the world is a student and is smart"

- Existential quantifiers are usually used with "and" to specify a list of properties about an individual:
  $(\exists x)$ student(x) $\wedge$ smart(x) means "There is a student who is smart"

- A common mistake is to represent this English sentence as the FOL sentence:
  $(\exists x)$ student(x) $\rightarrow$ smart(x)

  - Q. But what happens when there is a person who is *not* a student?

Quantifier Scope

- Switching the order of universal quantifiers *does not* change the meaning:
  - $(\forall x)(\forall y)\, P(x,y) \leftrightarrow (\forall y)(\forall x)\, P(x,y)$
  -
- Similarly, you can switch the order of existential quantifiers:
  - $(\exists x)(\exists y) P(x,y) \leftrightarrow (\exists y)(\exists x)\, P(x,y)$
  -
- Switching the order of universals and existentials *does* change meaning:
  - Everyone likes someone: $(\forall x)(\exists y)$ likes(x,y)
  - Someone is liked by everyone: $(\exists y)(\forall x)$ likes(x,y)

Connections between All and Exists

We can relate sentences involving $\forall$ and $\exists$ using De Morgan's laws: **(P(x)** as: **"(x) is perfect."**

1. $(\forall x) \neg P(x) \leftrightarrow \neg(\exists x) P(x)$
   a. "For all people, they are not perfect"
   b. "There is nobody who is perfect."

1. $\neg(\forall x) P(x) \leftrightarrow (\exists x) \neg P(x)$
   b. "It is not true that everyone is perfect"
   c. "There is at least one person who is imperfect."

1. $(\forall x) P(x) \leftrightarrow \neg (\exists x) \neg P(x)$
   b. "For all people, they are perfect"
   c. "There is no one who is imperfect."

1. $(\exists x) P(x) \leftrightarrow \neg(\forall x) \neg P(x)$
   b. "There exists a person who is perfect"
   c. "It is not true that everyone is not perfect."

- “Students who pass the course either do the homework or attend lecture;”
- “Bob did not attend every lecture;”
- “Bob passed the course.”

P(x)=Students pass the course

H(x)=Students do the homework

L(x)=Students attend lecture

Translate into logic as (with domain being students in the course):

- $\forall x(P(x) \rightarrow H(x) \vee L(x))$,
- $\neg L(b)$,
- $P(b)$.

1) Proof

a) Then we can conclude:

1. $\forall x(P(x) \rightarrow H(x) \vee L(x))$ [hypothesis]
2. $P(b) \rightarrow H(b) \vee L(b)$ [Universal instantiation]
3. $P(b)$ [hypothesis]
4. $H(b) \vee L(b)$ [modus ponens using (2) and (3)]
5. $\neg L(b)$ [hypothesis]
6. $H(b)$ [Disjunctive syllogism using (4) and (5)]

b) So, Bob must have done the homework.

Quantified inference rules

- Universal instantiation
  - $\forall x\ P(x) \therefore P(A)$
  -
- Universal generalization (Not Comes under inference rules in all circumstances)
  - $P(A) \wedge P(B) \ldots \therefore \forall x\ P(x)$
  -
- Existential instantiation
  - $\exists x\ P(x) \therefore P(F)$ ← **skolem constant F**
  -
- Existential generalization
  - $P(A) \therefore \exists x\ P(x)$

(Note : A constant that replaces an existentially quantified variable is called a Skolem constant)

Universal instantiation (a.k.a. universal elimination)

- If $(\forall x)$ P(x) is true, then P(C) is true, where C is *any* constant in the domain of x

- Example:

  $(\forall x)$ eats(Ziggy, x) $\Rightarrow$ eats(Ziggy, IceCream)

- The variable symbol can be replaced by any ground term, i.e., any constant symbol or function symbol applied to ground terms only

Universal generalization
(A.K.A. EXISTENTIAL INTRODUCTION)

- $P(A) \wedge P(B) \ldots \therefore \forall x\, P(x)$

The statement you provided represents a classic logical fallacy known as **hasty generalization** (or inductive fallacy).

In formal predicate logic, **this argument is invalid**.

Why it is Invalid

- **Premises:** (P(A)) and (P(B)) mean that object (A has property (P), and object (B) has property (P).
- **Conclusion:** (for all x P(x)) means *every single object* in the entire domain has property (P).

You cannot logically deduce that a property applies to **everything** just because it applies to a **few specific examples**.

Everyday Examples of this Fallacy

- **The Swan Example:**
  - *Premises:* Swan A is white. Swan B is white.
  - *Conclusion:* Therefore, **all swans are white**. (This is historically false; black swans exist).
- **The Coding Example:**
  - *Premises:* Laptop A runs Linux. Laptop B runs Linux.
  - *Conclusion:* Therefore, **every laptop in the world runs Linux**.

Existential instantiation (a.k.a. existential elimination)

- From $(\exists x)\, P(x)$ infer $P(c)$
- Example:
  - $(\exists x)\, \text{eats}(\text{Ziggy}, x) \rightarrow \text{eats}(\text{Ziggy}, \text{Stuff})$
- Note that the variable is replaced by a **brand-new constant** not occurring in this or any other sentence in the KB
- Also known as skolemization; constant is a **skolem constant**
- In other words, we don't want to accidentally draw other inferences about it by introducing the constant
- Convenient to use this to reason about the unknown object, rather than constantly manipulating the existential quantifier

Existential generalization (a.k.a. existential introduction)

- If P(c) is true, then $(\exists x)\ P(x)$ is inferred.
- Example

  $\text{eats(Ziggy, IceCream)} \Rightarrow (\exists x)\ \text{eats(Ziggy, x)}$

- All instances of the given constant symbol are replaced by the new variable symbol
- Note that the variable symbol cannot already exist anywhere in the expression

Knowledge Engineering in First-Order Logic (FOL)

Knowledge Engineering in First-Order Logic (FOL) involves encoding knowledge that is represented through objects, predicates, functions, and logical rules to enable AI systems to reason. It provides facts, axioms, and inference rules used to model complex relationships so machines can learn and make logical choices.

Applications of First-Order Logic in Knowledge Representation

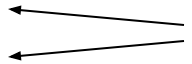
1. **Expert Systems**: FOL is used to represent expert knowledge in various domains such as medicine, finance, and engineering, enabling systems to reason and make decisions based on logical rules.
2. **Natural Language Processing**: FOL provides a formal framework for representing the meaning of natural language sentences, facilitating semantic analysis and understanding in NLP tasks.
3. **Semantic Web**: FOL is foundational to ontologies and knowledge graphs on the Semantic Web, enabling precise and machine-interpretable representations of knowledge.
4. **Robotics**: FOL is employed in robotic systems to represent spatial relationships, object properties, and task constraints, aiding in robot planning, navigation, and manipulation.
5. **Database Systems**: FOL-based query languages such as SQL enable expressive querying and manipulation of relational databases, allowing for complex data retrieval and manipulation.

Translating English to FOL

1. **Every gardener likes the sun.**
   $\forall x$ gardener(x) $\rightarrow$ likes(x,Sun)
1. **You can fool some of the people all of the time.**
   $\exists x \ \forall t$ person(x) $\wedge$ time(t) $\rightarrow$ can-fool(x,t)
1. **You can fool all of the people some of the time.**
   $\forall x \ \exists t$ (person(x) $\rightarrow$ time(t) $\wedge$ can-fool(x,t))
   $\forall x$ (person(x) $\rightarrow \ \exists t$ (time(t) $\wedge$ can-fool(x,t)))
1. **All purple mushrooms are poisonous**.
   $\forall x$ (mushroom(x) $\wedge$ purple(x)) $\rightarrow$ poisonous(x)



1. **No purple mushroom is poisonous.**
   $\neg \exists x$ purple(x) $\wedge$ mushroom(x) $\wedge$ poisonous(x)
   $\forall x$ (mushroom(x) $\wedge$ purple(x)) $\rightarrow \neg$poisonous(x)
1. **There are exactly two purple mushrooms**.
   $\exists x \ \exists y$ mushroom(x) $\wedge$ purple(x) $\wedge$ mushroom(y) $\wedge$ purple(y) ^ $\neg$(x=y) $\wedge \ \forall z$ (mushroom(z) $\wedge$ purple(z)) $\rightarrow$ ((x=z) $\vee$ (y=z))



1. **Clinton is not tall.**
   $\neg$tall(Clinton)
1. **X is above Y iff X is on directly on top of Y or there is a pile of one or more other objects directly on top of one another starting with X and ending with Y.**
   $\forall x \ \forall y$ above(x,y) $\leftrightarrow$ (on(x,y) $\vee \ \exists z$ (on(x,z) $\wedge$ above(z,y)))

Translating English to FOL

1. All students are intelligent.
2. Some students are not intelligent.
3. No teacher is a student.
4. Every dog is an animal.
5. Some animals are dangerous.
6. Every student likes a teacher.
7. Someone likes everyone.
8. Everyone likes someone.
9. No student likes every teacher.
10. There is exactly one student who is intelligent.
11.

Translating English to FOL

1. You can fool all of the people some of the time.
2. Every student admires some teacher who teaches every subject.
3. Some people trust everyone who trusts somebody.
4. Everyone who knows someone knows someone who is intelligent.
5. No one can fool everyone all of the time.
6. Every person has someone who loves them but whom they do not love.
7. Some teacher teaches every student who admires a teacher.
8. Everyone who likes all students dislikes at least one teacher.
9. There is someone whom everyone respects but who respects nobody.
10. Every person who trusts someone who trusts everyone trusts someone who trusts nobody

Example: A simple genealogy KB by FOL

- **Build a small genealogy knowledge base using FOL that**
  - contains facts of immediate family relations (spouses, parents, etc.)
  - contains definitions of more complex relations (ancestors, relatives)
  - is able to answer queries about relationships between people
- **Predicates:**
  - parent(x, y), child(x, y), father(x, y), daughter(x, y), etc.
  - spouse(x, y), husband(x, y), wife(x,y)
  - ancestor(x, y), descendant(x, y)
  - male(x), female(y)
  - relative(x, y)
- **Facts:**
  - husband(Joe, Mary), son(Fred, Joe)
  - spouse(John, Nancy), male(John), son(Mark, Nancy)
  - father(Jack, Nancy), daughter(Linda, Jack)
  - daughter(Liz, Linda)
  - etc.

- **Rules for genealogical relations**
  - $(\forall x,y)$ parent(x, y) $\leftrightarrow$ child (y, x)
    $(\forall x,y)$ father(x, y) $\leftrightarrow$ parent(x, y) $\wedge$ male(x) (similarly for mother(x, y))
    $(\forall x,y)$ daughter(x, y) $\leftrightarrow$ child(x, y) $\wedge$ female(x) (similarly for son(x, y))
  - $(\forall x,y)$ husband(x, y) $\leftrightarrow$ spouse(x, y) $\wedge$ male(x) (similarly for wife(x, y))
    $(\forall x,y)$ spouse(x, y) $\leftrightarrow$ spouse(y, x) (**spouse relation is symmetric**)
  - $(\forall x,y)$ parent(x, y) $\rightarrow$ ancestor(x, y)
    $(\forall x,y)(\exists z)$ parent(x, z) $\wedge$ ancestor(z, y) $\rightarrow$ ancestor(x, y)
  - $(\forall x,y)$ descendant(x, y) $\leftrightarrow$ ancestor(y, x)
  - $(\forall x,y)(\exists z)$ ancestor(z, x) $\wedge$ ancestor(z, y) $\rightarrow$ relative(x, y)
    (related by common ancestry)
    $(\forall x,y)$ spouse(x, y) $\rightarrow$ relative(x, y) (related by marriage)
    $(\forall x,y)(\exists z)$ relative(z, x) $\wedge$ relative(z, y) $\rightarrow$ relative(x, y) (**transitive**)
    $(\forall x,y)$ relative(x, y) $\leftrightarrow$ relative(y, x) **(symmetric**)
- **Queries**
  - ancestor(Jack, Fred) /* the answer is yes */
  - relative(Liz, Joe) /* the answer is yes */
  - relative(Nancy, Matthew)
    /* no answer in general, no if under closed world assumption */
  - $(\exists z)$ ancestor(z, Fred) $\wedge$ ancestor(z, Liz)

Summary of FOL Semantics

- A well-formed formula (“wff”) FOL is true or false with respect to a world and an interpretation (a model). The world has objects, relations, functions, and predicates. The interpretation maps symbols in the logic to the world.

Comparison Between Propositional Logic and Predicate Logic:

1. Expressiveness:

- **Propositional Logic**: Limited to dealing with simple, atomic propositions. It cannot express more complex statements like relationships between objects or quantification over variables. The propositions are either true or false without any further internal structure.

    - Example: "The sky is blue" is a simple proposition.
- **Predicate Logic (also known as First-Order Logic)**: More expressive. It allows for statements about objects, their properties, and their relationships using predicates, functions, and quantifiers. It can express relationships like "John is taller than Mary" or "All humans are mortal."

    - Example: "Human(x) → Mortal(x)" means "If x is a human, then x is mortal."

2. Syntax:

- **Propositional Logic**: Deals with propositional variables (such as P, Q, R) and logical connectives.

- **Predicate Logic**: Extends propositional logic by introducing predicates, variables, and quantifiers (such as $\forall$ for "all" and $\exists$ for "exists"). It allows reasoning about individual objects and their properties.

    - Example: $\forall x$ (Human(x) $\rightarrow$ Mortal(x)) says "For all x, if x is a human, then x is mortal."

3. Quantification:

- **Propositional Logic**: Cannot handle quantification. It only deals with fixed, known statements.

- **Predicate Logic**: Can handle quantification, meaning it can express statements like "For all" (universal quantifier, $\forall$) and "There exists" (existential quantifier, $\exists$).

    - Example: $\exists x$ (Human(x) $\wedge$ Tall(x)) means "There exists an x such that x is a human and x is tall."

4. Knowledge Representation:

- **Propositional Logic**: Best for simpler problems where facts can be represented as true/false propositions. It works well when there are no intricate relationships or properties to be expressed.

- **Predicate Logic**: Ideal for more complex problems where relationships between objects and the structure of the world need to be represented. It allows representing knowledge about individuals, relationships, and properties.

5. Example:

- **Propositional Logic**:
    - P: "The car is parked."
    - Q: "It is sunny."
    - Knowledge representation: $P \wedge Q$ (The car is parked and it is sunny.)
- **Predicate Logic**:
    - P(x): "x is a car."
    - Parked(x): "x is parked."
    - Sunny: "It is sunny."
    - Knowledge representation: $\exists x\,(P(x) \wedge Parked(x)) \wedge Sunny$ (There exists an x such that x is a car and x is parked, and it is sunny.)

Agents Based on Propositional Logic and First-Order Logic

In the field of **artificial intelligence (AI)**, **agents** are systems that perceive their environment, reason about it, and take actions to achieve goals.

The reasoning process of an agent is often based on **logical systems** such as **Propositional Logic (PL)** and **First-Order Logic (FOL)**, among others. These logical systems provide a formal foundation for representing knowledge, making decisions, and deriving conclusions.

1. Agents Based on Propositional Logic

In the case of **Propositional Logic**, an agent can reason with **propositions** that are either true or false. The knowledge of the agent is represented by a set of **propositions** (atomic sentences), and it uses **logical connectives** (AND, OR, NOT, etc.) to reason about the relationships between them.

How an Agent Uses Propositional Logic:

- **Perception**: The agent perceives certain facts about its environment as true or false (e.g., "It is raining", "The door is open").
- **Action Selection**: The agent can derive new facts based on existing knowledge and logical rules (e.g., "If it is raining, I should carry an umbrella").
- **Decision Making**: Based on the available knowledge and rules, the agent can decide the appropriate action to take.

Example:

- **Propositions**:
    - P: "It is raining."
    - Q: "I will carry an umbrella."
    - R: "I will stay inside."
- **Logical Rule**: "If it is raining, then I will carry an umbrella." This can be written as: P→Q(If P is true, then Q will be true).
- **Agent's Knowledge Base**: The agent's knowledge base might consist of several propositions and logical rules, such as:
  PPP: It is raining.
    - P→Q: If it rains, I will carry an umbrella.
- **Agent's Reasoning**:
    - Given that P (It is raining) is true, and P→Q (If it rains, I will carry an umbrella) is in the knowledge base, the agent can conclude that Q (I will carry an umbrella) is true.

Limitations of Propositional Logic for Agents:

- **Limited Expressiveness**: Propositional logic can only represent simple relationships and lacks the ability to express detailed knowledge about individual objects or more complex scenarios.
- **Scalability**: As the number of propositions increases, managing and reasoning about all the possible combinations can become cumbersome.

2. Agents Based on First-Order Logic (FOL)

First-Order Logic (FOL), also known as **Predicate Logic**, extends Propositional Logic by allowing for a more expressive representation of knowledge. It introduces **predicates**, **variables**, and **quantifiers** to represent and reason about objects, their properties, and relationships between them.

How an Agent Uses First-Order Logic:

- **Perception**: The agent perceives objects and their relationships (e.g., "John is a human", "John is taller than Mary").
- **Action Selection**: The agent uses logical rules to reason about these objects and relationships, making decisions based on more complex facts.
- **Decision Making**: The agent uses reasoning to infer new facts and take actions that are consistent with its goals.

Key Components of First-Order Logic for Agents:

- **Predicates**: Represent properties or relationships (e.g., Human(John),Taller(John,Mary)).
- **Variables**: Represent objects (e.g., x, y).
- **Quantifiers**: Express statements like "for all" (∀) or "there exists" (∃).

Example:

- **Predicates**:
  - Human(x): "x is a human."
  - Taller(x,y): "x is taller than y."
  - Umbrella(x): "x carries an umbrella."
- **Knowledge Base**:
  - Human(John): John is a human.
  - Taller(John,Mary): John is taller than Mary.
  - $\forall x(\text{Human}(x) \rightarrow \text{Mortal}(x))$: All humans are mortal.
- **Logical Rule**: "If it is raining, then I will carry an umbrella."
  - $\forall x(\text{Raining}(x) \rightarrow \text{Umbrella}(x))$ (For all x, if it is raining, then x will carry an umbrella).

Agent's Reasoning:

- The agent can reason about John by inferring that since John is a human (from Human(John)) and all humans are mortal (from $\forall x(\text{Human}(x) \rightarrow \text{Mortal}(x))$, it can conclude that John is mortal.
- If the agent has the rule $\forall x(\text{Raining}(x) \rightarrow \text{Umbrella}(x)$ and perceives Raining(John)to be true, it will infer that Umbrella(John)is true (John will carry an umbrella).

Advantages of Using First-Order Logic for Agents:

- **Expressiveness**: FOL allows for more complex reasoning, such as dealing with objects, relationships, and properties.
- **Quantification**: The use of quantifiers (∀ and ∃) enables the representation of statements about all or some objects in the domain.
- **Scalability**: While more complex, First-Order Logic provides greater scalability for reasoning about more complex environments.

Limitations:

- **Complexity**: FOL can be computationally expensive, especially when reasoning over large sets of objects and relations.
- **Inference**: The process of inference in FOL can become challenging as the complexity of the environment increases.

Example: Smart Home Agent

Imagine an agent in a **smart home** that can reason about the temperature, the state of windows, and other environmental factors to make decisions about heating or cooling.

1. **Propositional Logic** Agent (simpler):
    - Propositions:
        - P: "The window is open."
        - Q: "It is cold."
        - R: "Turn on the heater."
    - Rules: $P \rightarrow Q, Q \rightarrow R$
    - Knowledge: If the window is open, it is cold; if it is cold, turn on the heater.
2. **First-Order Logic** Agent (more advanced):
    - Predicates:
        - Open(x): "Window xxx is open."
        - Cold(x): "It is cold in room x."
        - TurnOnHeater(x): "Turn on the heater in room x."
    - Knowledge:
        - $\forall x(Open(x) \rightarrow Cold(x))$ (If a window is open, it is cold).
        - $\forall x(Cold(x) \rightarrow TurnOnHeater(x))$ (If it is cold, turn on the heater).
    - The agent can reason about multiple rooms, check the state of each window, and decide whether or not to turn on the heater based on the status of each room.

Reference

- **Text Books:**
  - Stuart Russell and Peter Norvig, “Artificial Intelligence: A Modern Approach”, Third edition, Pearson, 2003, ISBN :10: 0136042597

Thank You

Case Study

1) Medical Expert System – MYCIN
2) Knowledge-Based Reasoning in Intelligent Search Systems
3) AI-Based Rule Engine in Google Search

Case Study: Medical Expert System – MYCIN

1. Introduction

MYCIN was an early medical expert system developed at Stanford University in the 1970s. It was designed to help diagnose **bacterial infections** and recommend appropriate antibiotics.

MYCIN is a good example of **knowledge-based reasoning**, where knowledge is represented as facts and rules, and an inference engine uses those rules to reach conclusions.

2. MYCIN Using Propositional Logic (PL)

In **Propositional Logic**, statements are represented as propositions that are either true or false.

For example:

- F: Patient has fever
- I: Patient has an infection
- B: Patient has a bacterial infection
- A: Give antibiotic

A rule can be represented as:

$$F \land I \rightarrow B$$

and:

$$B \rightarrow A$$

If we know:

$$F$$

$$I$$

then using **Modus Ponens**:

$$F \land I$$

$$F \land I \rightarrow B$$

Therefore:

$$B$$

Then:

$$B \rightarrow A$$

Therefore:

$$A$$

So the system concludes that the patient should receive an antibiotic.

3. MYCIN Using First-Order Logic (FOL)

FOL allows us to represent **objects and relationships**, making it more expressive than PL.

Let:

- Patient(x): x is a patient
- Fever(x): x has fever
- Infection(x): x has an infection
- Bacterial(x): x has a bacterial infection
- NeedsAntibiotic(x): x needs an antibiotic

A general rule can be written:

$$\forall x[(\text{Fever}(x) \wedge \text{Infection}(x)) \rightarrow \text{Bacterial}(x)]$$

Another rule:

$$\forall x[\text{Bacterial}(x) \rightarrow \text{NeedsAntibiotic}(x)]$$

Facts: Fever(John)

Infection(John)

Inference

From: $\text{Fever(John)} \wedge \text{Infection(John)}$

and

$(\text{Fever}(x) \wedge \text{Infection}(x)) \rightarrow \text{Bacterial}(x)$

we infer: Bacterial(John) (Modes Pones)

Then: $\text{Bacterial(John)} \rightarrow \text{NeedsAntibiotic(John)}$

Therefore: NeedsAntibiotic(John)

4. Inference Rules Used

1. Modus Ponens

The most basic inference rule is:

$P, P \rightarrow Q$

Therefore:

Q

Example:

Bacterial(John)

Bacterial(John)→NeedsAntibiotic(John)

Therefore:

NeedsAntibiotic(John)

2. **Universal Instantiation**

From:

$$\forall x(Bacterial(x) \rightarrow NeedsAntibiotic(x))$$

we can substitute John:

$$Bacterial(John) \rightarrow NeedsAntibiotic(John)$$

3. Forward Chaining

MYCIN-style rule systems can reason **forward**:

Facts→Rules→New Facts→Conclusion

Example:

Fever(John)

Infection(John)

↓

Bacterial(John)

↓

NeedsAntibiotic(John)

Conclusion

MYCIN demonstrates how AI can use FOL, PL, facts, rules, and inference mechanisms to perform expert reasoning. A knowledge base stores facts and rules, while an inference engine applies rules such as Modus Ponens, Universal Instantiation, forward chaining, and backward chaining to derive new knowledge. Similar principles of knowledge representation and reasoning contribute to modern intelligent search systems, although systems such as Google Search use many additional technologies beyond classical FOL and PL.

Complete it

1) Knowledge-Based Reasoning in Intelligent Search Systems
2) AI-Based Rule Engine in Google Search